

EV Powertrain Simulator

Complete Project Notes & Concept Reference

MATLAB (Code) + Simulink (Block Diagram) Implementation

Personal reference notes covering Phases 1-4 of the MATLAB simulation and Phases 1-3 of the Simulink model. Includes every core formula, the reasoning behind each design choice, real bugs encountered and how they were diagnosed, and the specific questions asked along the way with their explanations.

Contents

- 1. Project Overview
- 2. Phase 1 (MATLAB): Battery SOC Model - Coulomb Counting
- 3. Phase 2 (MATLAB): Driving Profile & Regenerative Braking
- 4. Phase 3 (MATLAB): Motor Model, Throttle Control & Vehicle Dynamics
- 5. Phase 4 (MATLAB): Signal Processing - Noise, FFT & Filtering
- 6. Simulink Phase 1: Battery SOC Block Model
- 7. Simulink Phase 2: Driving Profile with Signal Editor
- 8. Simulink Phase 3: Motor Model in Blocks
- 9. Master Formula Sheet
- 10. Key Debugging Lessons (Cross-Phase)

1. Project Overview

This project simulates the core systems of an electric vehicle from first principles: the battery's state of charge, a driving profile with regenerative braking, the motor's electrical behaviour, a simple throttle controller, vehicle dynamics, and signal processing on simulated sensor data. It was built twice — once as MATLAB code (four phases) and once as an equivalent Simulink block diagram (three phases) — to understand the same physics from two different angles.

Why build it this way

Instead of copying a template, each phase was built by starting from the governing physical equation, translating it into code/blocks, running it, and then debugging whatever looked physically wrong using reasoning rather than guesswork. Every bug found along the way (SOC going negative, phantom idle current, unit mismatches, wrongly-placed blocks) taught a real modelling lesson, not just a syntax fix.

Core system parameters used throughout

Symbol	Meaning	Value used
Q	Battery capacity	2 Ah (Phase 1-2) -> 20 Ah (from Phase 3 onward)
SOC0	Initial state of charge	100%
V	Motor supply voltage	48 V
R	Motor winding resistance	0.5 Ohm
Kt	Torque constant	0.5 Nm/A
Ke	Back-EMF constant	0.5 V.s/rad
r_wheel	Wheel radius	0.3 m
m	Vehicle mass	1200 kg
I_discharge_max	Max discharge current	50 A
I_charge_max	Max charge/regen current	15 A

2. Phase 1 (MATLAB): Battery SOC Model

2.1 Core concept: State of Charge

SOC is simply how full the battery is, expressed as a percentage — a fuel gauge for charge. 100% = full, 0% = empty.

2.2 Coulomb Counting

The method used to estimate SOC: track how much current has flowed out of the battery over time, and subtract that from the starting charge. Current x time = charge, measured in Coulombs (or, at the larger scale used here, Amp-hours).

$$\text{SOC}(t) = \text{SOC}_0 - (100/Q) \times \int I(\tau) d\tau$$

Discretised for a simulation loop (Euler integration - update SOC a little bit at every small time step Δt):

$$\text{SOC}[k+1] = \text{SOC}[k] - (I[k] \times \Delta t / Q) \times 100$$

2.3 The MATLAB implementation, block by block

```
Q = 2.0;           % battery capacity (Ah)
SOC0 = 100;        % initial SOC (%)
I_load = 1.0;      % constant discharge current (A)

dt = 1/60;         % time step in HOURS (1 minute)
t_end = 2;         % simulate 2 hours
t = 0:dt:t_end;
N = length(t);

SOC = zeros(1, N); % preallocate
SOC(1) = SOC0;     % set starting value

for k = 1:N-1
    SOC(k+1) = SOC(k) - (I_load * dt / Q) * 100;
end
```

2.4 Doubts raised and how they were resolved

My question: What is 'preallocating' an array, and why bother with `SOC = zeros(1,N)`?

An empty array of the correct final size (N slots, all zero) is created up front, then filled in one value at a time. This is far faster than letting the array silently grow one element per loop iteration, because MATLAB does not need to resize/copy memory on every step. `SOC(1) = SOC0` fills in only the very first slot with the real starting charge; the rest stay as placeholder zeros until the loop fills them in.

My question: Coulomb counting - where does the name and the units come from?

A Coulomb is the unit of electric charge: 1 Coulomb = 1 Amp flowing for 1 second. `I_load * dt` in the code literally calculates how much charge (in Amp-hours here) left the battery in that one time step. Summing that over every loop iteration is 'counting the coulombs' that flowed out - hence the name.

This is genuinely how real Battery Management Systems estimate SOC, usually combined with a voltage-based correction to fight long-run drift.

My question: *Why did the SOC graph go below 0% (down to -150%) when Q and I_load were changed?*

The loop had no rule telling it to stop once SOC reached 0 - it just kept subtracting blindly. Fix: clamp the value inside the loop so it can never leave the 0-100% range, exactly like a real Battery Management System does:

```
for k = 1:N-1
    SOC(k+1) = SOC(k) - (I_load * dt / Q) * 100;
    if SOC(k+1) < 0
        SOC(k+1) = 0;
    elseif SOC(k+1) > 100
        SOC(k+1) = 100;
    end
end
```

My question: *MATLAB error: 'Unable to run file... file names must start with an alphabetic character...'*

Caused by a space in the filename ('battery soc model.m'). MATLAB script filenames cannot contain spaces - renamed to battery_soc_model.m (underscore instead of space) and it ran fine.

My question: *The script ran with no errors, but no graph appeared - why?*

The plot had actually been created, but the Figure window opened behind the main MATLAB window / off-screen. Typing figure(1) in the Command Window, or checking the taskbar/Alt+Tab, brings the existing figure to the front.

2.5 Result

Constant 1 A draw on a 2 Ah battery discharges linearly from 100% to 0% over exactly 2 hours — matching the hand-calculated prediction and confirming the model is correct.

3. Phase 2 (MATLAB): Driving Profile & Regenerative Braking

3.1 The upgrade from Phase 1

I_load becomes a full array instead of one fixed number, built with an if/elseif chain that checks the current simulation time and assigns a different current for each phase of a simple drive cycle:

- 0-15 min: acceleration, current ramps 0 → 5 A
- 15-35 min: cruising, steady 3 A
- 35-45 min: braking / regen, current drops to -1 A (charging)
- 45-60 min: idle, 0 A

3.2 Sign convention (the key idea of this phase)

Positive current = discharging | **Negative current = charging**

Because $SOC(k+1) = SOC(k) - (I \times dt/Q) \times 100$, feeding in a negative I makes the formula **subtract a negative number**, which is mathematically identical to **adding** to SOC. The same single equation handles both discharging and charging - no separate 'if charging' branch needed, purely because of the sign convention.

3.3 Doubts raised and how they were resolved

My question: *How does a battery actually get charged (physically), especially during braking?*

Charging/discharging are the same internal chemical reaction running in opposite directions (like compressing vs. releasing a spring). During regenerative braking, the spinning wheels turn the motor backwards, and a motor spun without power applied acts as a generator - it produces electricity instead of consuming it. That generated electricity is pushed back into the battery instead of being wasted as heat in traditional friction brakes. Our simple model does not simulate the chemistry itself, only the direction and amount of current, and assumes 100% charging efficiency as a simplification (a real limitation, noted for later refinement).

My question: *Why does the SOC dip only slightly upward during the braking phase, not by a lot?*

Braking only lasts 10 of the 60 minutes, and the regen current (-1 A) is much smaller than the peak discharge current (+5 A) - so only a small fraction of the used energy is recovered, matching how real-world regenerative braking behaves (recovers a modest fraction of energy used, not a full recharge).

3.4 Result

SOC drops fastest during acceleration and cruise, ticks upward slightly during the braking window, then stays flat during idle — the shape used as the reference target for every later Simulink and Phase 3 rebuild.

4. Phase 3 (MATLAB): Motor Model, Throttle & Vehicle Dynamics

4.1 The big shift

Instead of hand-picking current values (Phase 2), current is now **calculated** from motor physics and vehicle motion: Vehicle motion → motor torque needed → current the battery must supply.

4.2 Core motor equations

Torque: $T = K_t \times I$

Back-EMF: $V_{emf} = K_e \times \omega$

Current (Ohm's law with back-EMF opposing supply voltage):

$I = (V - K_e \times \omega) / R$

Physical meaning: a spinning motor generates its own opposing voltage (back-EMF). The faster it spins, the more it 'pushes back' against the battery, so less current flows at high speed and more current flows near zero speed (this is why real motors draw their highest 'stall current' at rest).

4.3 Throttle / controller layer

A raw physics equation with full voltage always applied is unrealistic - it implies the motor draws maximum current even when the car is stationary and the driver's foot is off the pedal. A throttle signal (-1 to 1) representing driver intent is multiplied onto the raw current:

$I_{raw} = (V - K_e \times \omega) / R$ $I_{load} = throttle \times I_{raw}$

- throttle = 1 → full accelerator pressed, apply full current
- throttle = 0.3 → light throttle, just enough to cruise
- throttle = -1 → full regenerative braking
- throttle = 0 → pedal released / idle, zero current demanded

Note: Throttle is dimensionless (no physical unit) - it is a pure scaling ratio, like a volume knob. Multiplying Amps by a unitless number between -1 and 1 still gives a result in Amps; it only scales the magnitude.

4.4 Realistic current limits (asymmetric)

Real batteries can discharge quickly but can only accept charge slowly (like a wide drain but a narrow fill funnel on a tank). Two separate current caps were applied instead of one:

```
I_discharge_max = 50;    % Amps
I_charge_max = 15;       % Amps (lower - batteries accept charge slower)

I_load(I_load > I_discharge_max) = I_discharge_max;
I_load(I_load < -I_charge_max) = -I_charge_max;
```

4.5 Vehicle dynamics: torque -> force -> acceleration

$$F = T / r_{\text{wheel}} \quad a = F / m \quad (\text{Newton's second law})$$

Dividing torque by wheel radius is the same lever/wrench principle as turning a bolt: a smaller wheel radius converts the same torque into more force at the road contact point; a larger wheel converts it into less force. Newton's second law then converts that force into acceleration, given the vehicle's mass.

4.6 Doubts raised and how they were resolved

My question: Why did idle current come out to 96 A, and why is that wrong?

At $\omega = 0$ (car stationary), $I = (48 - 0.5 \times 0) / 0.5 = 96 \text{ A}$ - the equation has no concept of 'driver's foot off the pedal', it just applies full battery voltage regardless of whether the car is moving. A real parked car draws close to 0 A. This 96 A 'phantom current' during a 15-minute idle phase is exactly what crashed SOC to 0% - fixed by the throttle layer (throttle=0 during idle forces $I_{\text{load}} = 0$ regardless of I_{raw}).

My question: With the symmetric +/-50 A limit, why did SOC jump all the way back to 100% during braking?

Charge added = $I \times t = 50 \text{ A} \times (10/60 \text{ hr}) \approx 8.3 \text{ Ah}$, while the battery was only $Q = 2 \text{ Ah}$ - more charge than the whole battery could hold, so it saturated instantly. Root cause traced with the actual numbers rather than assumed to be a bug in the clamp itself.

My question: After lowering the charge limit to 15 A, SOC still saturated to 100% - why?

$15 \text{ A} \times (10/60 \text{ hr}) = 2.5 \text{ Ah}$, still more than the 2 Ah battery capacity. The real issue was a scale mismatch between battery size and motor current, not the current limit. Fixed by increasing Q from 2 Ah to a more realistic 20 Ah, after which the same currents represent a sensible fraction of total capacity and SOC changes gradually and realistically.

My question: How exactly was the 'average current' during the acceleration ramp (2.5 A) calculated?

For a straight-line ramp, average = (start value + end value)/2 = $(0+5)/2 = 2.5 \text{ A}$. This is the same result as computing the area under the ramp (a triangle: $0.5 \times \text{base} \times \text{height}$) and dividing by time - both give the same total charge used, since charge = current $\times$ time = area under the current-vs-time graph.

4.7 Result

A fully self-consistent chain: throttle $\rightarrow$ current $\rightarrow$ acceleration $\rightarrow$ SOC, all derived from real physics equations with no arbitrary hand-picked current values anywhere in the chain.

5. Phase 4 (MATLAB): Signal Processing - Noise, FFT & Filtering

5.1 Why this phase exists

Real sensors never report perfectly clean values. Phase 4 adds realistic noise to the clean current signal, then uses frequency-domain analysis (connecting directly to DTFT coursework) and filtering to recover a clean signal from a noisy one - the exact problem those transform tools are built to solve.

5.2 Adding sensor noise

Measured signal = True signal + Noise

```
noise_amplitude = 2; % Amps
sensor_noise = noise_amplitude * randn(1, N);
I_measured = I_load + sensor_noise;
```

randn generates Gaussian (normal-distribution) random numbers, chosen because real combined sensor noise (thermal/Johnson noise, electromagnetic interference from nearby motor currents, analog-to-digital quantization error, mechanical vibration, power-supply ripple) statistically tends toward a Gaussian shape when many small independent effects add together (Central Limit Theorem).

5.3 Frequency-domain analysis (FFT)

FFT decomposes a time-domain signal into the frequency components that make it up - the same core idea as DTFT, just computed efficiently for a finite number of samples.

```
Fs = 1/dt; % sampling frequency
Y = fft(I_measured);
P2 = abs(Y/N);
P1 = P2(1:floor(N/2)+1);
P1(2:end-1) = 2*P1(2:end-1);
f = Fs*(0:floor(N/2))/N; % frequency axis
```

Result: a tall spike near the low frequency that matches how often the whole drive cycle repeats (true signal - slow, smooth) and a spread-out, roughly flat noise floor across the rest of the spectrum (random noise - no dominant frequency). The x-axis naturally caps at $F_s/2$, the Nyquist frequency - the fastest frequency detectable at that sampling rate.

5.4 Moving average filter

$y[n] = (1/M) \times \sum x[n-k] \quad \text{for } k = 0 \text{ to } M-1$

```
M = 3; % window size
I_filtered = movmean(I_measured, M);
```

This is a real FIR filter (Finite Impulse Response), described by a difference equation - directly connected to Z-transform coursework. Averaging cancels out random noise (positive and negative

jitters partially cancel) while preserving the underlying trend, because nearby true signal values are similar (the signal changes slowly) while nearby noise values are uncorrelated.

5.5 The window-size trade-off (confirmed by direct comparison)

Smaller M (e.g. 2-3)	Larger M (e.g. 7)
Less noise cancelled	More noise cancelled, smoother output
More responsive - tracks sharp changes (e.g. the acceleration ramp) closely	Less responsive - blurs/lags behind sharp changes, rounds off corners

5.6 Doubts raised and how they were resolved

My question: *What exactly does 'window size' mean in a moving average filter?*

At each output point, the filter averages together that sample plus the (M-1) samples right before it, then slides forward by one sample each step - always using the M most recent values. Worked through with a concrete 7-sample numeric example to show noisy values (e.g. 42, 38, 45, 40...) averaging down to a much steadier output.

My question: *Is the FFT-verification / Z-transform connection step essential, or optional?*

Confirmed as optional academic polish, not essential to the working simulator: the practical workflow (simulate noise, understand its source, see it in frequency domain, build and tune a working filter) was already complete. FFT-verifying the filter and writing its formal $H(z)$ transfer function are valuable for exam prep but don't change the project itself.

6. Simulink Phase 1: Battery SOC Block Model

6.1 Why Simulink

Simulink is a visual, block-diagram simulation tool. Instead of writing a for loop, blocks are connected and Simulink runs the time-stepping internally. It is the industry-standard tool for this kind of system-level automotive/EV simulation.

6.2 Block-to-equation mapping

MATLAB code concept	Simulink block
I_load (current source)	Constant block
-100/Q scaling	Gain block
Running integral over time (the for loop)	Integrator block (1/s)
SOC0 (starting value)	Integrator's 'Initial condition' field
if SOC<0 / SOC>100 clamp	Saturation block (limits 0 to 100)
plot(t, SOC)	Scope block

6.3 Doubts raised and how they were resolved

My question: *Why is the Gain block set to a negative value (-50)?*

The Integrator block can only ADD (accumulate) whatever signal flows into it - it has no built-in subtraction. Since discharging must reduce SOC, the Gain feeds in a negative number so that 'adding a negative' has the same net effect as subtracting. With $Q=2$, $\text{Gain} = -100/Q = -50$. (If current were negative, i.e. charging, the double negative would correctly produce a positive addition to SOC - matching Phase 2's regen behaviour.)

My question: *Why did SOC crash to 0% within the first ~60-100 seconds instead of taking the full hour?*

A units mismatch: the Gain (-50) was calculated assuming the Integrator's time base was in HOURS (matching Phase 1's Stop Time = 2). Once the model switched to a Signal-Editor-based driving profile defined in SECONDS with Stop Time = 3600, the Integrator was now accumulating Amp-seconds, not Amp-hours - so the old Gain over-scaled everything by a factor of 3600. Fixed by dividing the gain by 3600: new Gain = $-100/Q/3600$. Same lesson as dt needing to be in hours back in Phase 1 MATLAB - Simulink does not automatically convert time units between blocks.

7. Simulink Phase 2: Driving Profile with Signal Editor

7.1 Signal Editor

Signal Builder was renamed 'Signal Editor' in newer MATLAB/Simulink versions. It lets a custom time-varying signal be defined as an explicit TIME/DATA table of points, connected by straight lines - a visual, numeric-entry equivalent of the if/elseif profile-building code from MATLAB Phase 2.

7.2 Building sharp transitions

To create an instantaneous jump (e.g. current suddenly dropping from 3 A to -1 A at the moment braking starts), two points are entered at the exact same time value with two different data values - the straight-line interpolation between them becomes a vertical drop.

TIME	DATA	
0	0	
900	5	
900	3	<- two points at t=900 = sharp drop
2100	3	
2100	-1	<- two points at t=2100 = sharp drop
2700	-1	
2700	0	<- two points at t=2700 = sharp rise
3600	0	

7.3 Doubts raised and how they were resolved

My question: *Between 900s and 2100s the graph sloped down from 5 to 3 instead of staying flat at 3 - why?*

Signal Editor draws a straight line directly between consecutive points. Going straight from (900, 5) to (2100, 3) naturally slopes between them since nothing was holding the value at 5 right after t=900. Fixed by adding an extra point (900, 3) immediately after (900, 5), forcing an instant drop, followed by a flat line to (2100, 3).

My question: *Error: 'untitled.mat does not exist on the MATLAB path' when running the model.*

The Signal Editor's drawn scenario only lives in the editor window until it is explicitly saved to a .mat file (Ctrl+S). The Signal Editor block on the canvas was pointing at a file name that had never actually been saved. Fixed by saving the scenario (e.g. driving_profile.mat) in the same folder as the model, and confirming the block's 'File name' field pointed to it.

8. Simulink Phase 3: Motor Model in Blocks

8.1 Block chain implementing the motor equation

$$I_{\text{raw}} = (V - K_e \times \omega) / R \quad I_{\text{load}} = \text{throttle} \times I_{\text{raw}}$$

Block (renamed)	Purpose	Value
speed profile1	Signal Editor: motor speed omega(t)	trapezoid: 0->100->100->0->0 rad/s
ke (Gain)	$K_e \times \omega$	0.5
v-kw (Sum, +/-)	$V - K_e \times \omega$	inputs: Constant 48 and ke output
res (Gain)	divide by R ($1/R$)	2 (since $R = 0.5$)
throttle profile	Signal Editor: driver intent	1 / 0.3 / -1 / 0 by phase
x (Product)	$I_{\text{load}} = \text{throttle} \times I_{\text{raw}}$	-
limit (Saturation)	clamp real current	Upper 50, Lower -15
Gain (-K-)	convert current to SOC-rate	$-100/Q/3600$, $Q=20$
1/s (Integrator)	accumulate SOC over time	Initial condition 100
Saturation	clamp SOC	Upper 100, Lower 0

8.2 Worked numeric example ($t=0$, $\omega=0$)

$$I_{\text{raw}} = (48 - 0.5 \times 0) \times 2 = 96 \text{ A (the motor's 'stall current')}$$

8.3 Doubts raised and how they were resolved

My question: How exactly is I_{raw} calculated block by block, tracing the actual values used?

speed profile1 outputs ω -> Gain 'ke' (0.5) computes $K_e \times \omega$ -> Constant(48) and that result feed into Sum 'v-kw' configured +/- to compute $V - K_e \times \omega$ -> Gain 'res' (2, i.e. $1/R$) multiplies to complete $I_{\text{raw}} = (V - K_e \times \omega)/R$. This then flows into the Product block alongside the throttle profile to produce the final I_{load} .

My question: After adding a Saturation block set to +50/-15, the graph did not change at all - why?

The Saturation block had been connected AFTER the SOC-rate Gain block instead of right after the Product (current) output. By that point the signal was already a tiny SOC-rate number (around -0.0014), nowhere near the +50/-15 limits, so the clamp was doing nothing. Fixed by reordering the chain: Product -> Saturation (clamps the real current) -> Gain (converts the already-clamped current to SOC rate) -> Integrator. This matched exactly how MATLAB Phase 3 clamped I_{load} itself before ever computing SOC change - a lesson that block ORDER matters just as much as block VALUES.

My question: With $Q=2$ in Simulink, the SOC curve crashed to 0% almost instantly with the motor model - why?

Same battery-vs-current scale mismatch discovered in MATLAB Phase 3: motor stall current can reach 96 A, far too large relative to a 2 Ah battery. Fixed by using $Q=20$ in the Simulink Gain

calculation as well (-100/20/3600), matching the MATLAB fix.

8.4 Result

Final SOC curve: steep drop during acceleration, gentler decline during cruise, a modest realistic uptick during the braking/regen phase (not a full recharge, thanks to the correctly-placed asymmetric current limits), then flat during idle - closely matching the MATLAB Phase 3 result and confirming both implementations agree.

9. Master Formula Sheet

Concept	Formula
Battery SOC (Coulomb counting)	$SOC(t) = SOC_0 - (100/Q) \times \int I \, dt$
Discretised SOC update	$SOC[k+1] = SOC[k] - (I[k] \times \Delta t / Q) \times 100$
Motor torque	$T = K_t \times I$
Back-EMF	$V_{emf} = K_e \times \omega$
Motor current (with back-EMF)	$I = (V - K_e \times \omega) / R$
Throttle-scaled current	$I_{load} = throttle \times I_{raw}$
Force at wheel	$F = T / r_{wheel}$
Newton's second law	$a = F / m$
Charge from constant current	$Charge \, (Ah) = I \times t$
Average of a linear ramp	$Average = (start \, value + end \, value) / 2$
Measured sensor signal	$Measured = True \, signal + Noise$
Moving average filter (FIR)	$y[n] = (1/M) \times \sum x[n-k], \text{ for } k = 0..M-1$
Nyquist frequency	$f_{max_detectable} = F_s / 2$

10. Key Debugging Lessons (Cross-Phase)

These are the recurring, transferable lessons that showed up across both MATLAB and Simulink, worth remembering beyond this specific project:

- **Always add a physical stopping condition.** A raw equation will happily produce impossible results (SOC below 0%, above 100%) unless a clamp/saturation is explicitly added.
- **Units must match everywhere, every time.** dt in hours vs. Q in Amp-hours (MATLAB); Integrator time base in seconds vs. a Gain calculated for hours (Simulink) - both caused results to be wrong by a clean factor (3600) that was traceable once suspected.
- **Block/step ORDER matters as much as block VALUES.** A correctly-configured Saturation block did nothing because it was placed after the value had already been scaled down to a tiny number.
- **Check the actual numbers before assuming a limit is 'artificial'.** The SOC plateauing at ~20-25% was not a bug or a hidden cap - it was the correct result of the specific driving cycle's net charge usage, confirmed by hand-calculating charge used vs. charge recovered.
- **Scale mismatches are easy to miss.** A 2 Ah battery combined with 15-50 A currents meant charge added/removed in minutes could exceed the entire battery capacity - fixed by using a more realistic capacity (20 Ah) rather than changing the currents.
- **A model without a controller is not the same as a model with a controller.** Raw motor physics alone implied maximum current even at a standstill (96 A phantom idle current) - a throttle/driver-intent layer was needed to represent real-world behaviour.

End of notes. This document reflects the reasoning built up phase by phase, in the order the project was actually built and debugged, rather than a cleaned-up final summary - so it works as a genuine 'why did I do this' reference when revisiting the project later.